Árvore Binária de Busca

Árvore Binária de Busca

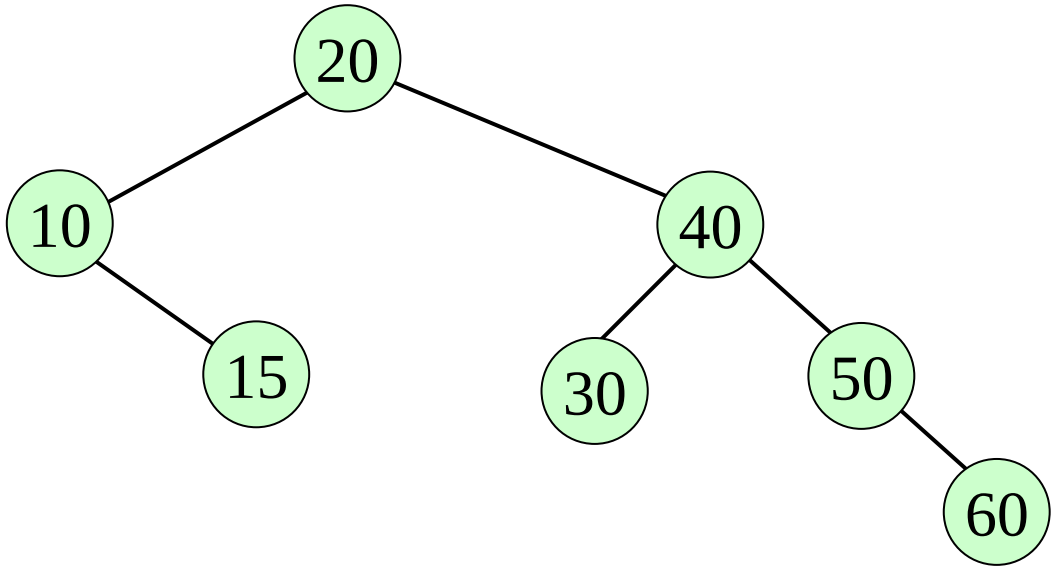
Uma árvore binária de busca é uma árvore binária tal que:

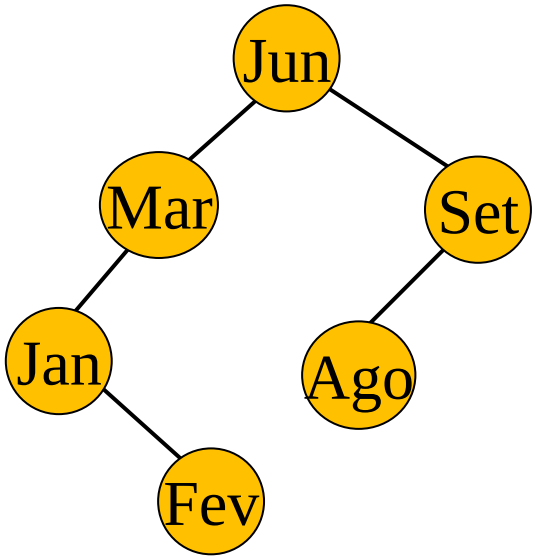
i) A raiz possui uma chave
ii) As chaves dos nós da subárvore esquerda da raiz são menores que a chave da raiz.
iii) As chaves dos nós da subárvore direita da raiz são maiores que a chave da raiz.
iv) As subárvores esquerda e direita são árvores binárias de busca



Árvore Binária de Busca

Exemplos





Árvore Binária de Busca

Operações básicas:

- **Busca**
- **Inserção**
- **Remoção**

Árvore Binária de Busca

Busca

Considere $S = \{s_1,...,s_n\}$ um conjunto de n chaves tais que $s_1 < ... < s_n$.

Dado um valor x, verificar se $x \in S$.

Caso $x = s_i$, $s_i \in S$, retornar i.

Árvore Binária de Busca

Qual a complexidade da busca em um lista linear?

Qual a complexidade da busca em uma árvore binária?

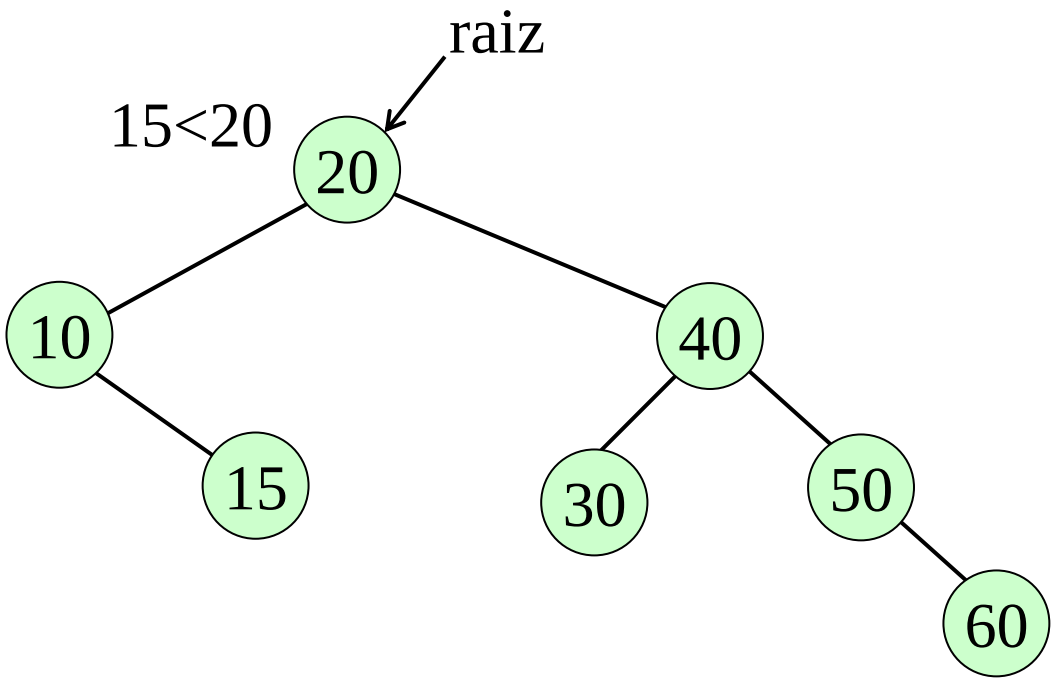
Qual a complexidade da busca em uma árvore binária de busca?



Árvore Binária de Busca

Exemplo Busca

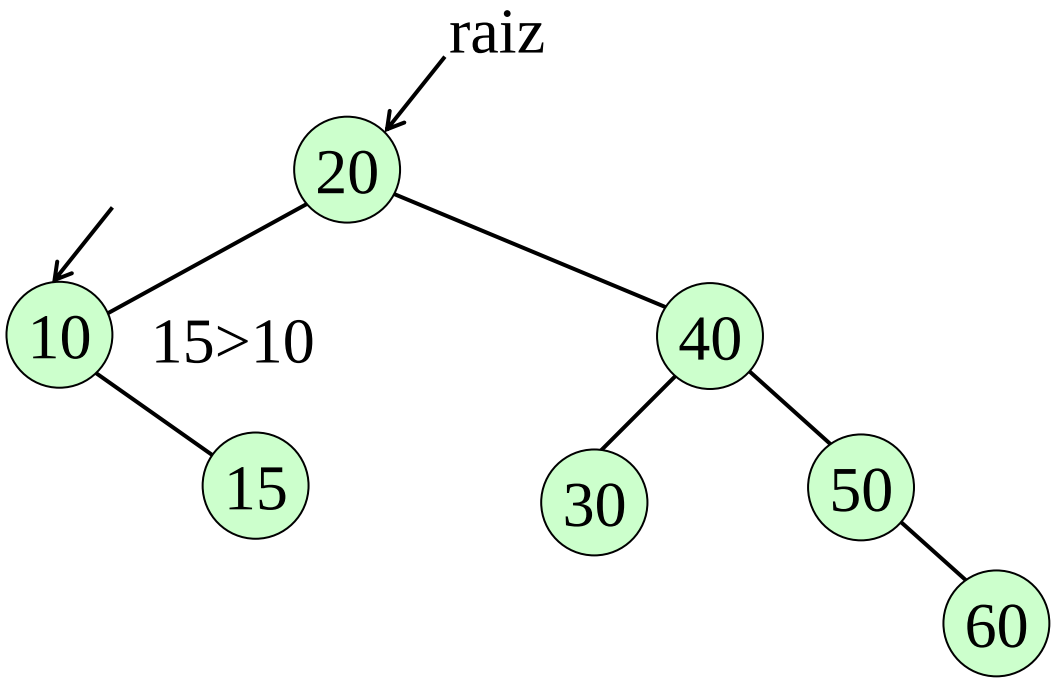
Chaves de busca: 15



Árvore Binária de Busca

Exemplo Busca

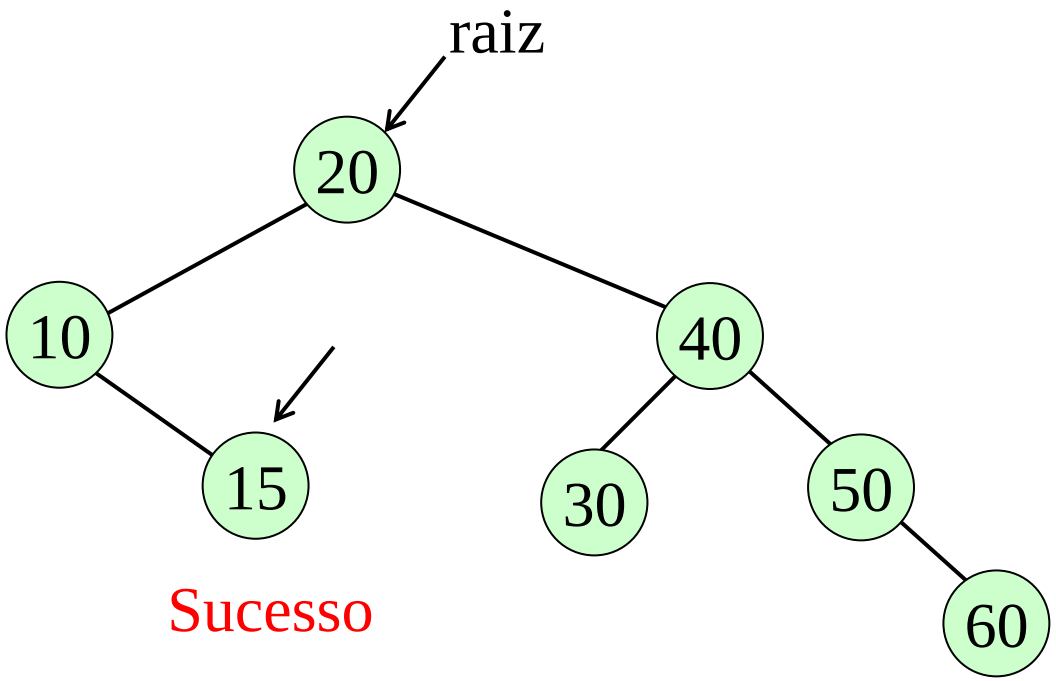
Chaves de busca: 15



Árvore Binária de Busca

Exemplo Busca

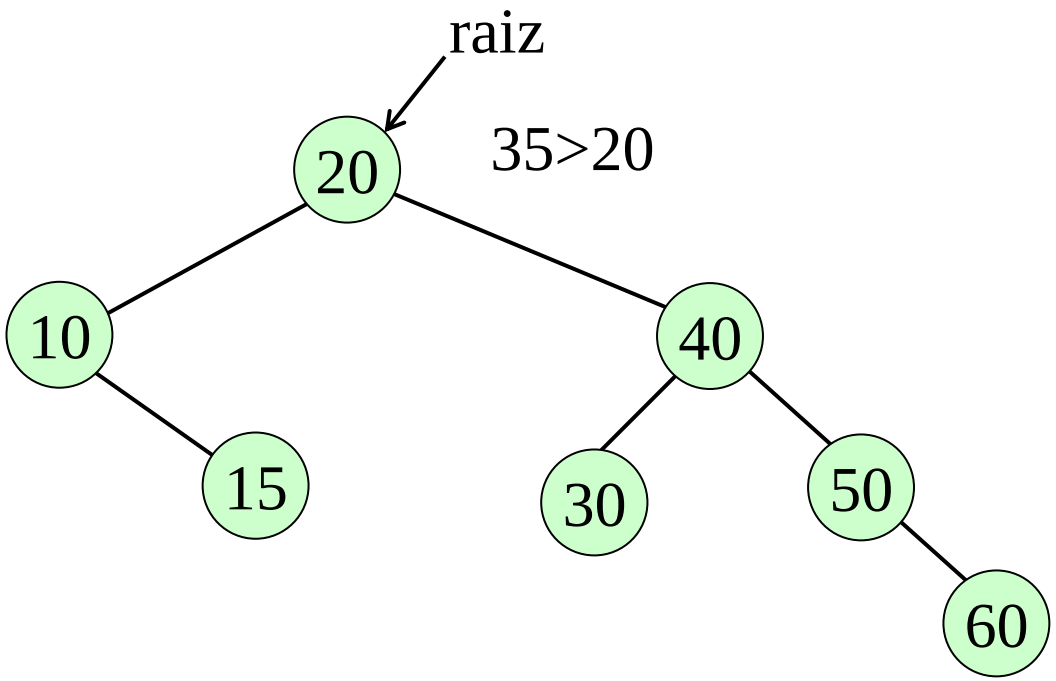
Chaves de busca: 15



Árvore Binária de Busca

Exemplo Busca

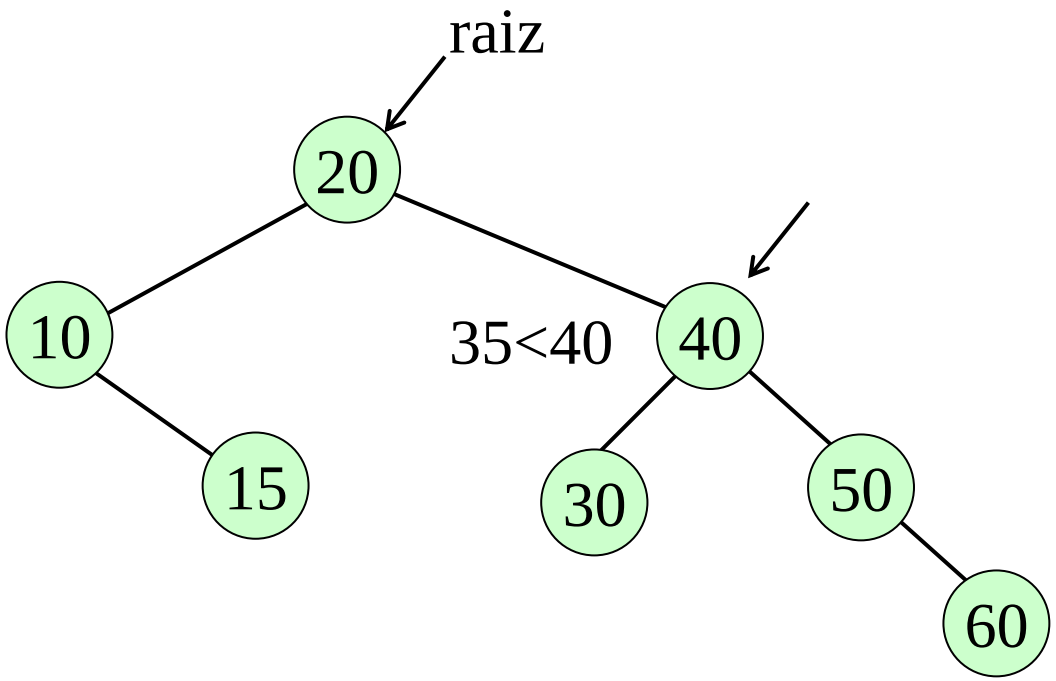
Chaves de busca: 35



Árvore Binária de Busca

Exemplo Busca

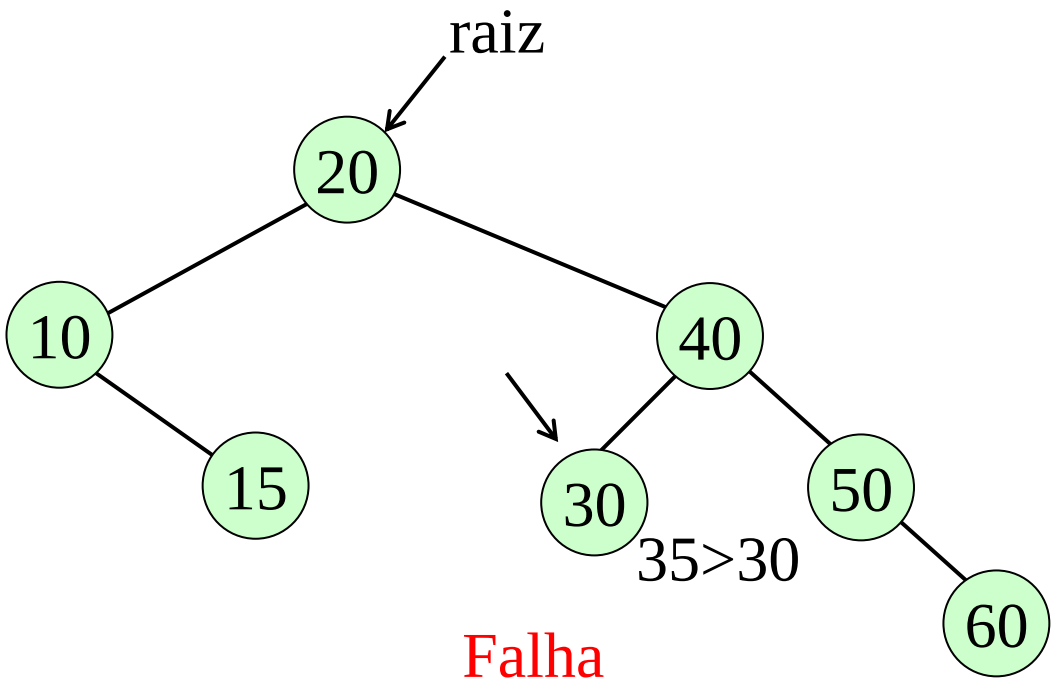
Chaves de busca: 35



Árvore Binária de Busca

Exemplo Busca

Chaves de busca: 35



Árvore Binária de Busca

Algoritmo Busca

Busca_arvbb(raiz, *x*, 0)

***f* =** 0, árvore vazia

***f* =** 1, chave encontrada e *pt* aponta para nó onde a chave se encontra

***f* =** 2, chave não encontrada e *pt* aponta para nó cuja árvore esquerda é vazia

***f* =** 3, chave não encontrada e *pt* aponta para nó cuja árvore direita é vazia

Árvore Binária de Busca

Algoritmo Busca

Busca_arvbb(raiz, *x*, 0)

```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

***f* = 0**, árvore vazia

***f* = 1**, chave encontrada e *pt* aponta para nó onde a chave se encontra

***f* = 2**, chave não encontrada e *pt* aponta para nó cuja árvore esquerda é vazia

***f* = 3**, chave não encontrada e *pt* aponta para nó cuja árvore direita é vazia

Árvore Binária de Busca

Algoritmo Busca

Busca_arvbb(raiz, *x*, 0)

```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

***f* = 0**, árvore vazia

***f* = 1**, chave encontrada e *pt* aponta para nó onde a chave se encontra

***f* = 2**, chave não encontrada e *pt* aponta para nó cuja árvore esquerda é vazia

***f* = 3**, chave não encontrada e *pt* aponta para nó cuja árvore direita é vazia

Árvore Binária de Busca

Algoritmo Busca

Busca_arvbb(raiz, *x*, 0)

```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

***f* = 0**, árvore vazia

***f* = 1**, chave encontrada e *pt* aponta para nó onde a chave se encontra

***f* = 2**, chave não encontrada e *pt* aponta para nó cuja árvore esquerda é vazia

***f* = 3**, chave não encontrada e *pt* aponta para nó cuja árvore direita é vazia

Árvore Binária de Busca

Algoritmo Busca

Busca_arvbb(raiz, *x*, 0)

```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

***f* = 0**, árvore vazia

***f* = 1**, chave encontrada e *pt* aponta para nó onde a chave se encontra

***f* = 2**, chave não encontrada e *pt* aponta para nó cuja árvore esquerda é vazia

***f* = 3**, chave não encontrada e *pt* aponta para nó cuja árvore direita é vazia

Árvore Binária de Busca

Algoritmo Busca

Busca_arvbb(raiz, *x*, 0)

```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

***f* = 0**, árvore vazia

***f* = 1**, chave encontrada e *pt* aponta para nó onde a chave se encontra

***f* = 2**, chave não encontrada e *pt* aponta para nó cuja árvore esquerda é vazia

***f* = 3**, chave não encontrada e *pt* aponta para nó cuja árvore direita é vazia

Árvore Binária de Busca

Algoritmo Busca

Busca_arvbb(raiz, *x*, 0)

```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

***f* = 0**, árvore vazia

***f* = 1**, chave encontrada e *pt* aponta para nó onde a chave se encontra

***f* = 2**, chave não encontrada e *pt* aponta para nó cuja árvore esquerda é vazia

***f* = 3**, chave não encontrada e *pt* aponta para nó cuja árvore direita é vazia

Árvore Binária de Busca

Algoritmo Busca

Busca_arvbb(raiz, *x*, 0)

```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

***f* = 0**, árvore vazia

***f* = 1**, chave encontrada e *pt* aponta para nó onde a chave se encontra

***f* = 2**, chave não encontrada e *pt* aponta para nó cuja árvore esquerda é vazia

***f* = 3**, chave não encontrada e *pt* aponta para nó cuja árvore direita é vazia

Árvore Binária de Busca

Complexidade da Busca

Melhor Caso:

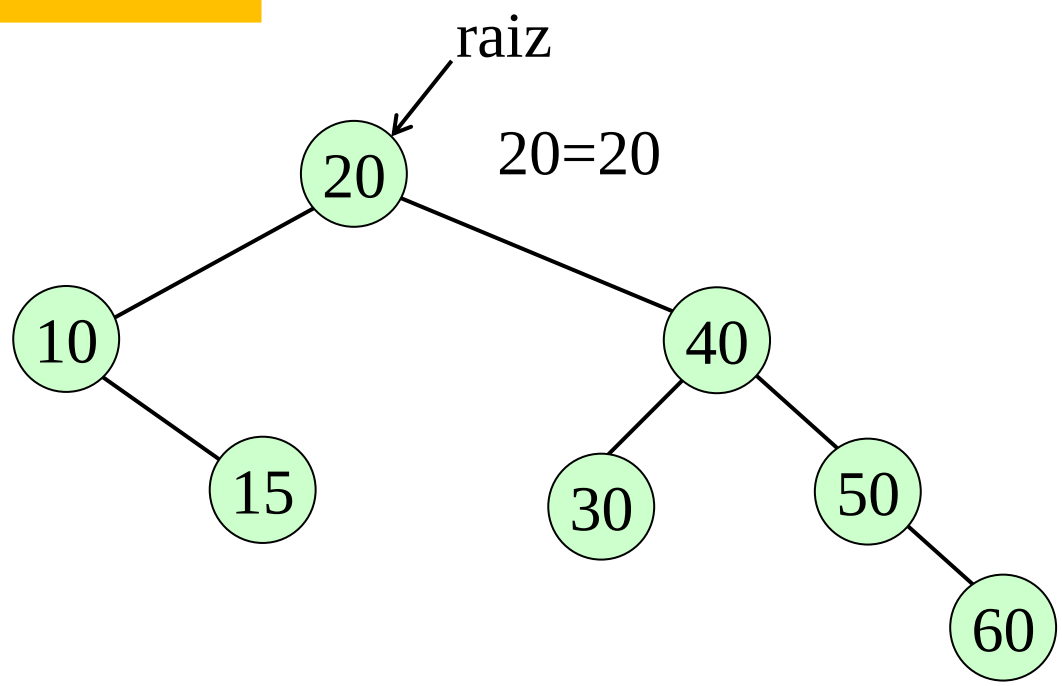


Árvore Binária de Busca

Complexidade da Busca

Melhor Caso: Chave é encontrada na raiz

Exemplo Busca: 20





Árvore Binária de Busca

Complexidade da Busca

Pior Caso:



Árvore Binária de Busca

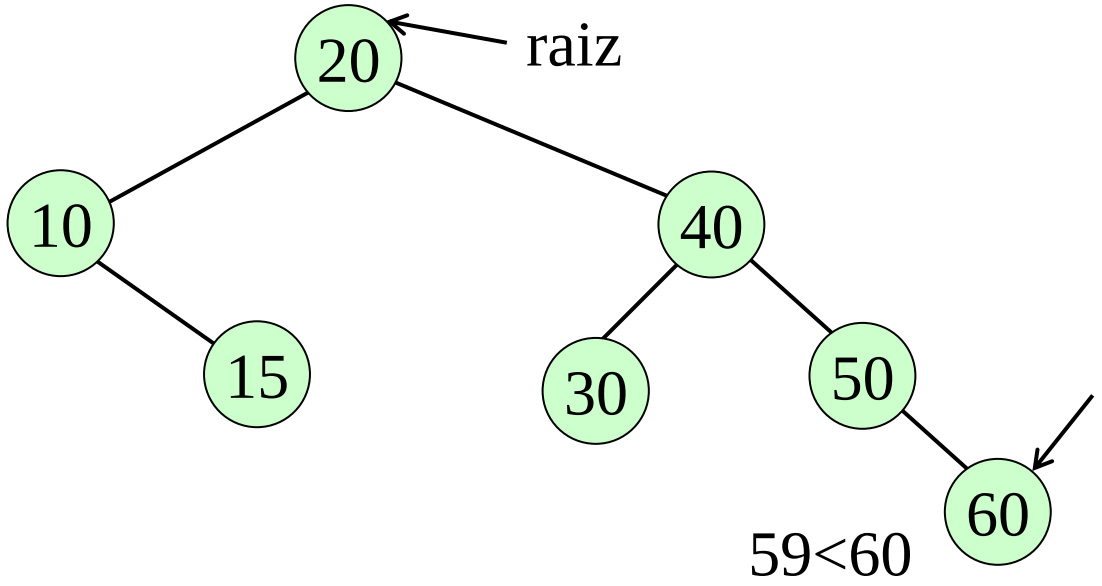
Complexidade da Busca

Pior Caso: Chave encontrada ou não encontrada no maior caminho entre a raiz e uma folha

=

***h*(*T*)**

Exemplo Busca: 59



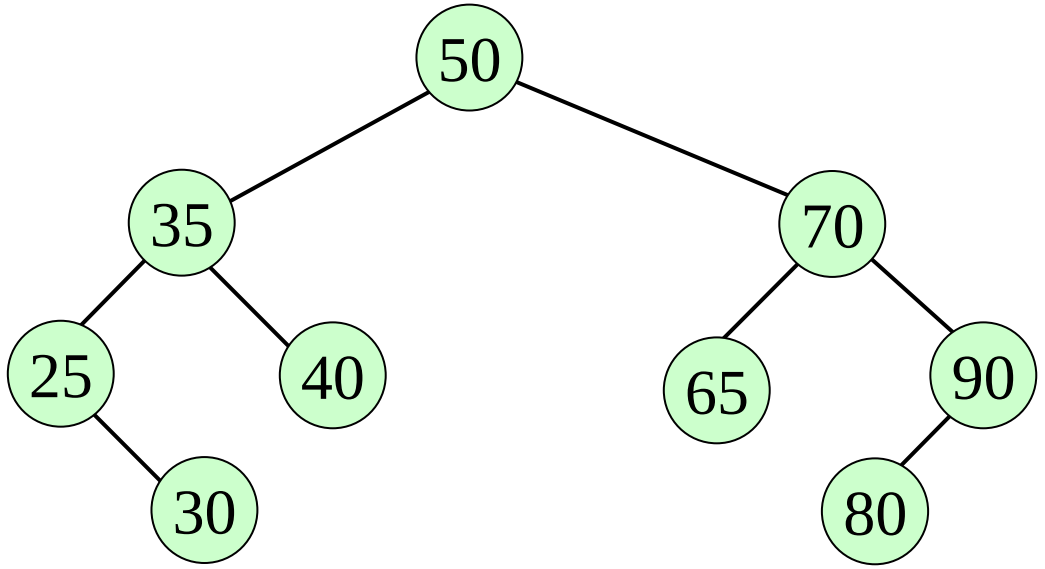
Árvore Binária de Busca

Quais são os casos para $h(T)$?

Árvore Binária de Busca

Árvore Binária Completa



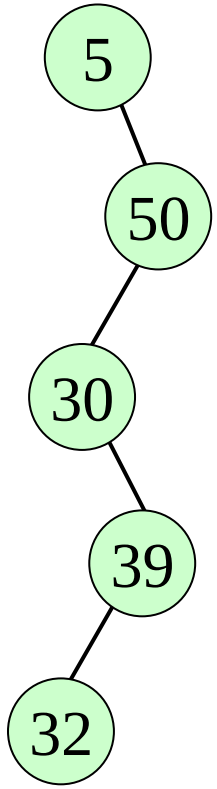


Lema 2

Considere T uma árvore binária completa com $n > 0$ nós.
Então T possui altura h mínima.
Além disso, $h = 1 + \lfloor \log n \rfloor$.

Árvore Binária de Busca

Árvore Zig-zag



$$h(T) = O(n)$$

Árvore Binária de Busca

Complexidade da Busca

A complexidade da busca depende da altura da árvore.

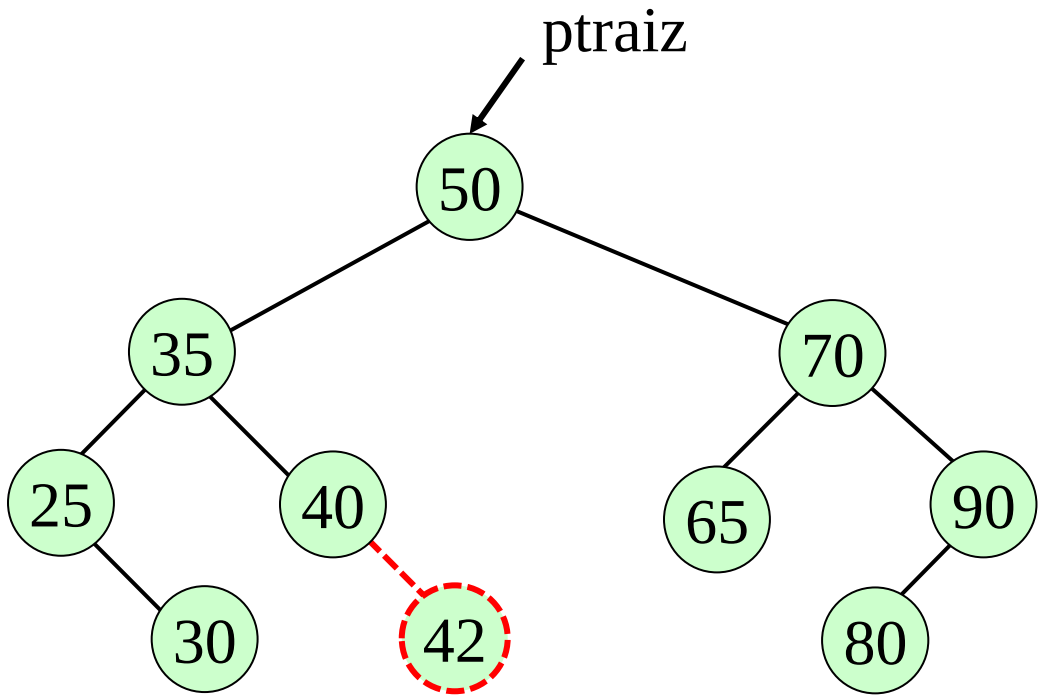
Sendo assim, é interessante construir a árvore com a menor altura possível.

Como visto anteriormente, a árvore que possui altura mínima para um conjunto de *n* chaves é a *árvore completa*. Nesse caso, a complexidade do algoritmo é *O*(log *n*).

Árvore Binária de Busca

Inserção

O novo nó é inserido no lugar de uma subárvore vazia.



Árvore Binária de Busca

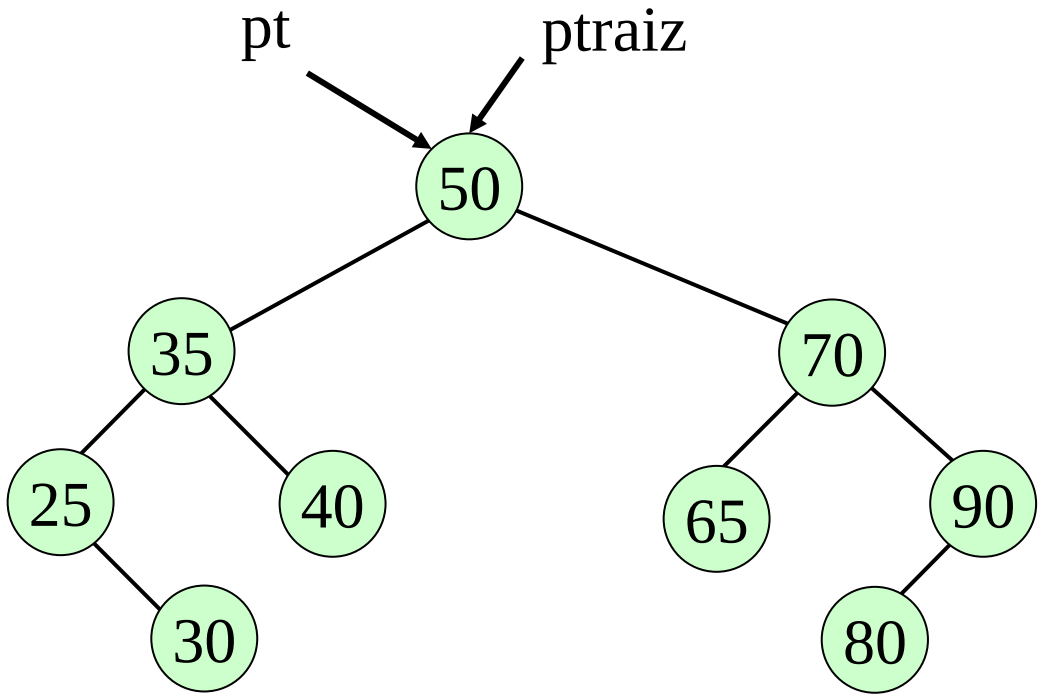
Inserção

O novo nó é inserido no lugar de uma subárvore vazia.

```
inserção(x, pt, f)
    pt ← ptraiz
    busca_árvore(x, pt, f)
    se (f = 1) então "Elemento já existe"
    senão
        ocupar(pt1); pt1↑.chave ← x; pt1↑.esq = λ; pt1↑.dir= λ
        se (f = 0) então ptraiz ← pt1
        senão
           se (f = 2) então pt↑.esq = pt1
           senão pt↑.dir = pt1
```

Exemplo Inserção

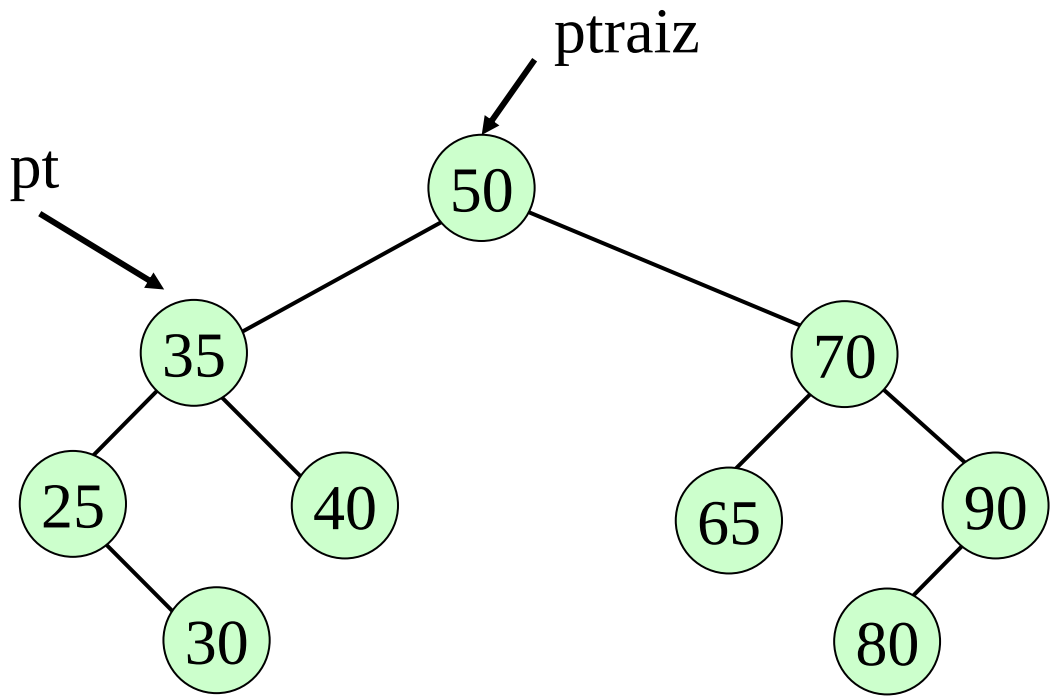
x=42




```
inserção(x, pt, f)
    pt ← ptraiz
    busca_árvore(x, pt, f)
    se (f = 1) então "Elemento já existe"
    senão
        ocupar(pt1); pt1↑.chave ← x; pt1↑.esq = λ; pt1↑.dir= λ
        se (f = 0) então ptraiz ← pt1
        senão
          se (f = 2) então pt↑.esq = pt1
          senão pt↑.dir = pt1
```

Exemplo Inserção

x=42

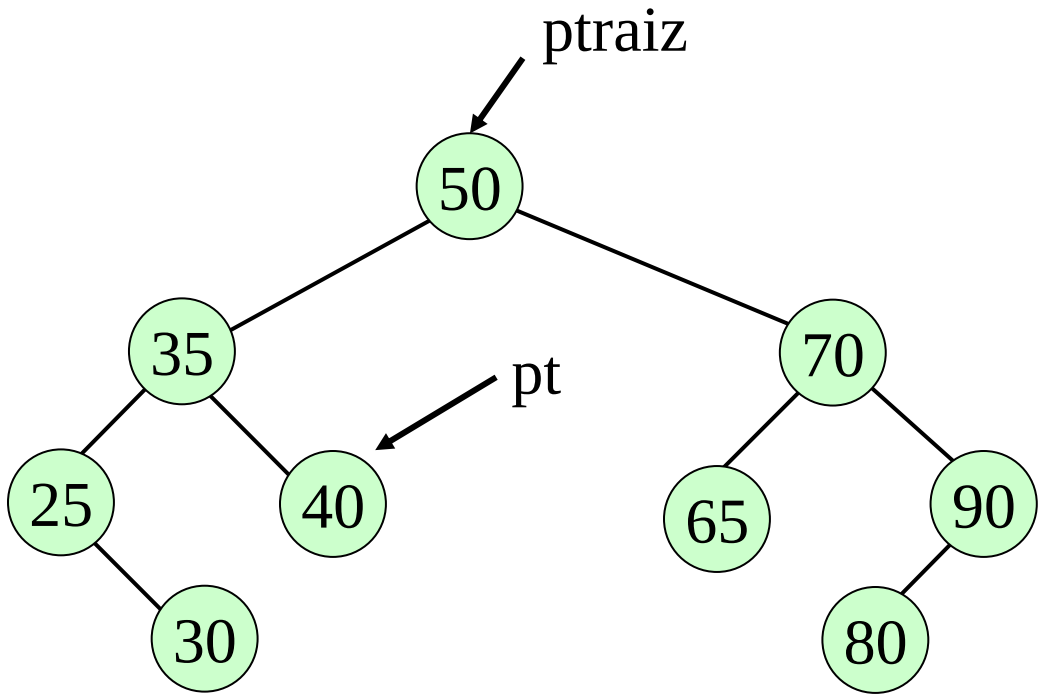



```
inserção(x, pt, f)
    pt ← ptraiz
    busca_árvore(x, pt, f)
    se (f = 1) então "Elemento já existe"
    senão
        ocupar(pt1); pt1↑.chave ← x; pt1↑.esq = λ; pt1↑.dir= λ
        se (f = 0) então ptraiz ← pt1
        senão
          se (f = 2) então pt↑.esq = pt1
          senão pt↑.dir = pt1
```

```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

Exemplo Inserção

x=42



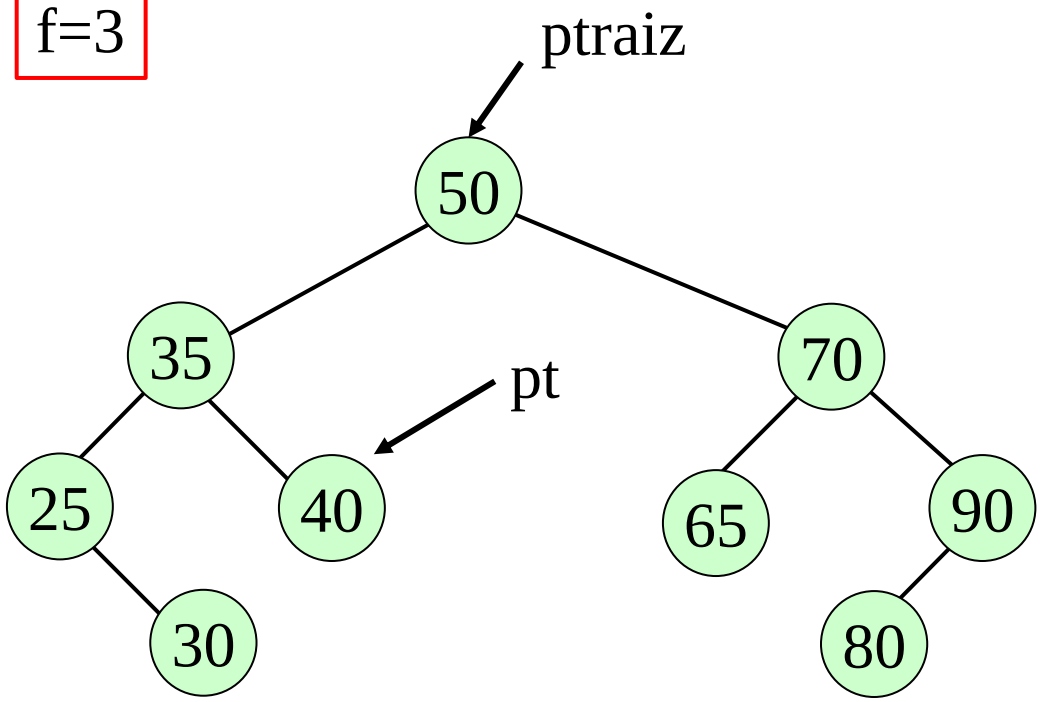

```
inserção(x, pt, f)
   pt ← ptraiz
   busca_árvore(x, pt, f)
   se (f = 1) então "Elemento já existe"
   senão
      ocupar(pt1); pt1↑.chave ← x; pt1↑.esq = λ; pt1↑.dir= λ
      se (f = 0) então ptraiz ← pt1
      senão
        se (f = 2) então pt↑.esq = pt1
        senão pt↑.dir = pt1
```

```
Busca_arvbb(pont_no pt, int x, int f)
 se (pt ≠ λ)
   se (pt↑.chave = x) f ← 1
   senão
     se (pt↑.chave > x)
       se (pt↑.esq = λ) f ← 2
       senão pt ← pt↑.esq
     senão
       se (pt↑.dir = λ) f ← 3
       senão pt ← pt↑.dir
     se (f < 1)
       Busca_arvbb(pt, x, f)
```

Exemplo Inserção

x=42

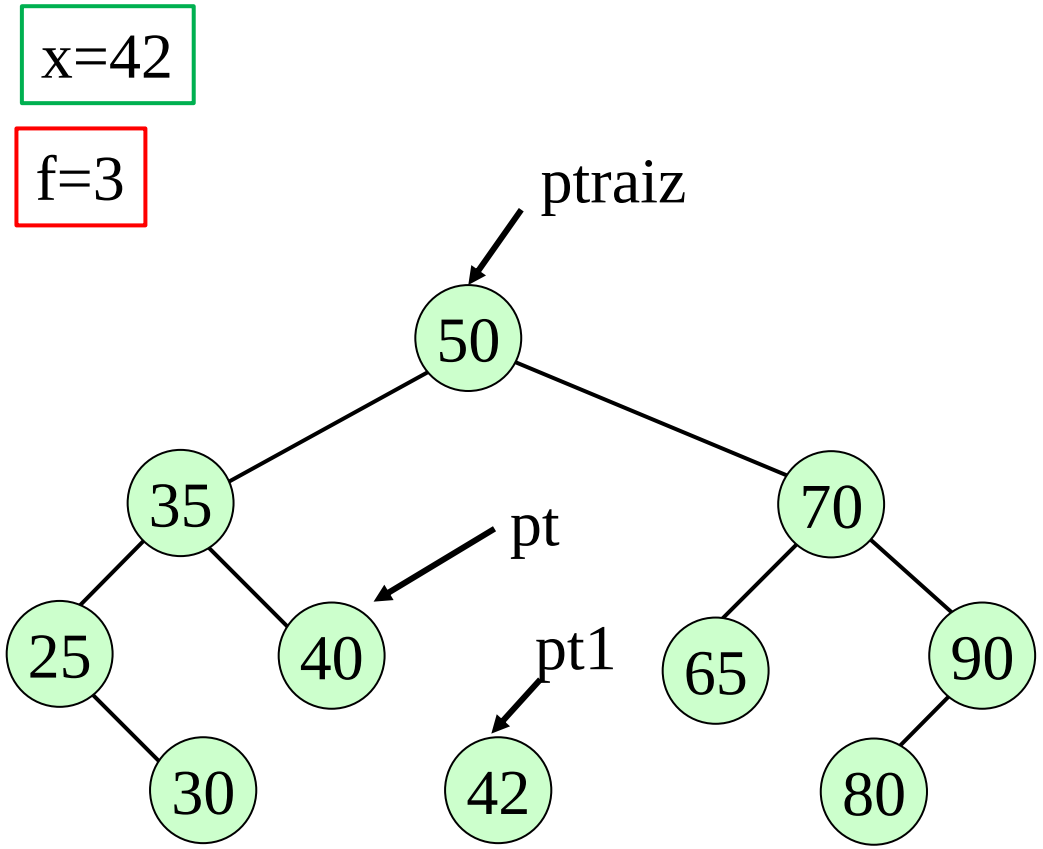
f=3




```
inserção(x, pt, f)
   pt ← ptraiz
   busca_árvore(x, pt, f)
   se (f = 1) então "Elemento já existe"
   senão
      ocupar(pt1); pt1↑.chave ← x; pt1↑.esq = λ; pt1↑.dir= λ
      se (f = 0) então ptraiz ← pt1
      senão
        se (f = 2) então pt↑.esq = pt1
        senão pt↑.dir = pt1
```

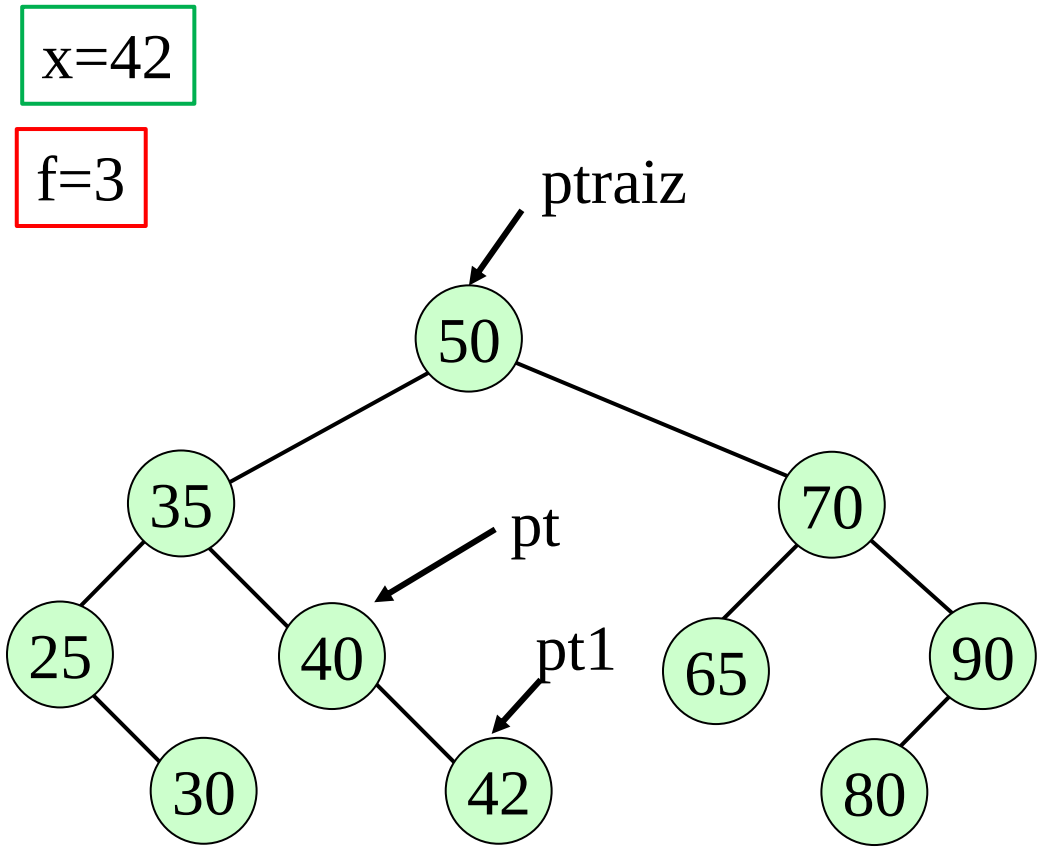
```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

Exemplo Inserção




```
inserção(x, pt, f)
    pt ← ptraiz
    busca_árvore(x, pt, f)
    se (f = 1) então "Elemento já existe"
    senão
        ocupar(pt1); pt1↑.chave ← x; pt1↑.esq = λ; pt1↑.dir= λ
        se (f = 0) então ptraiz ← pt1
        senão
            se (f = 2) então pt↑.esq = pt1
            senão pt↑.dir = pt1
```

Exemplo Inserção

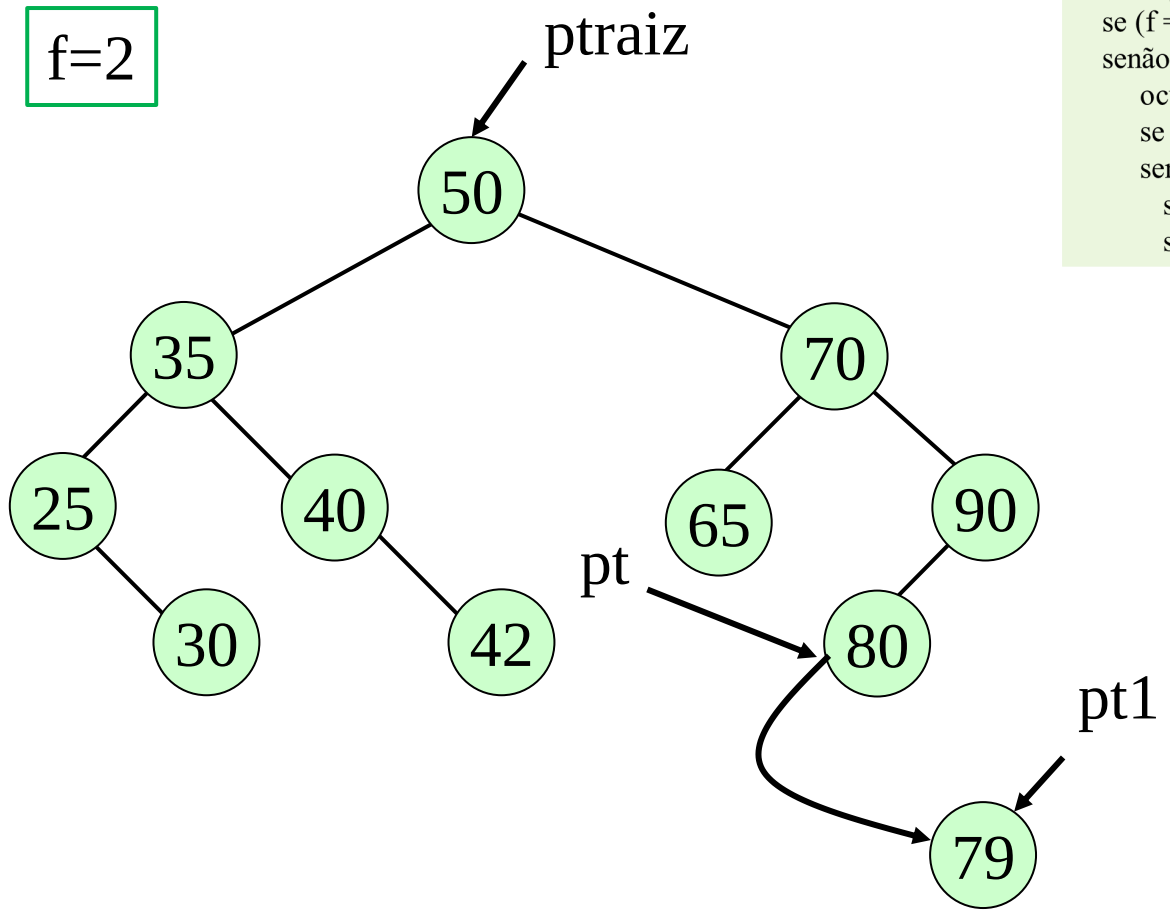



```
inserção(x, pt, f)
    pt ← ptraiz
    busca_árvore(x, pt, f)
    se (f = 1) então "Elemento já existe"
    senão
        ocupar(pt1); pt1↑.chave ← x; pt1↑.esq = λ; pt1↑.dir= λ
        se (f = 0) então ptraiz ← pt1
        senão
          se (f = 2) então pt↑.esq = pt1
          senão pt↑.dir = pt1
```

Exemplo Inserção

x=79

f=2




```
inserção(x, pt, f)
   pt ← ptraiz
   busca_árvore(x, pt, f)
   se (f = 1) então "Elemento já existe"
   senão
       ocupar(pt1); pt1↑.chave ← x; pt1↑.esq = λ; pt1↑.dir= λ
       se (f = 0) então ptraiz ← pt1
       senão
          se (f = 2) então pt↑.esq = pt1
          senão pt↑.dir = pt1
```

```
Busca_arvbb(pont_no pt, int x, int f)
  se (pt ≠ λ)
    se (pt↑.chave = x) f ← 1
    senão
      se (pt↑.chave > x)
        se (pt↑.esq = λ) f ← 2
        senão pt ← pt↑.esq
      senão
        se (pt↑.dir = λ) f ← 3
        senão pt ← pt↑.dir
      se (f < 1)
        Busca_arvbb(pt, x, f)
```

Árvore Binária de Busca

Inserção

Complexidade?

```
inserção(x, pt, f)
    pt ← ptraiz
    busca_árvore(x, pt, f)
    se (f = 1) então "Elemento já existe"
    senão
        ocupar(pt1); pt1↑.chave ← x; pt1↑.esq = λ; pt1↑.dir= λ
        se (f = 0) então ptraiz ← pt1
        senão
           se (f = 2) então pt↑.esq = pt1
           senão pt↑.dir = pt1
```

Árvore Binária de Busca

Inserção

Complexidade?

Dominada pela complexidade da busca

Exercício

Dada uma lista com *n* chaves, construir a árvore de busca binária de altura mínima utilizando o algoritmo de inserção.

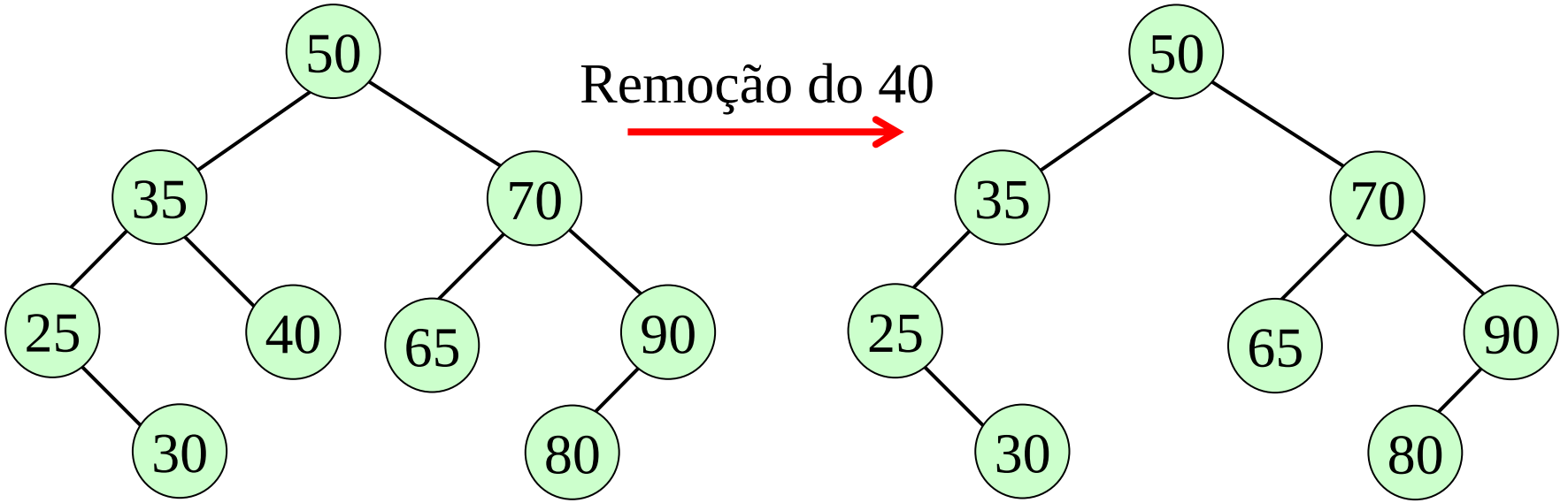
Qual a complexidade do seu método?



Árvore Binária de Busca

Remoção

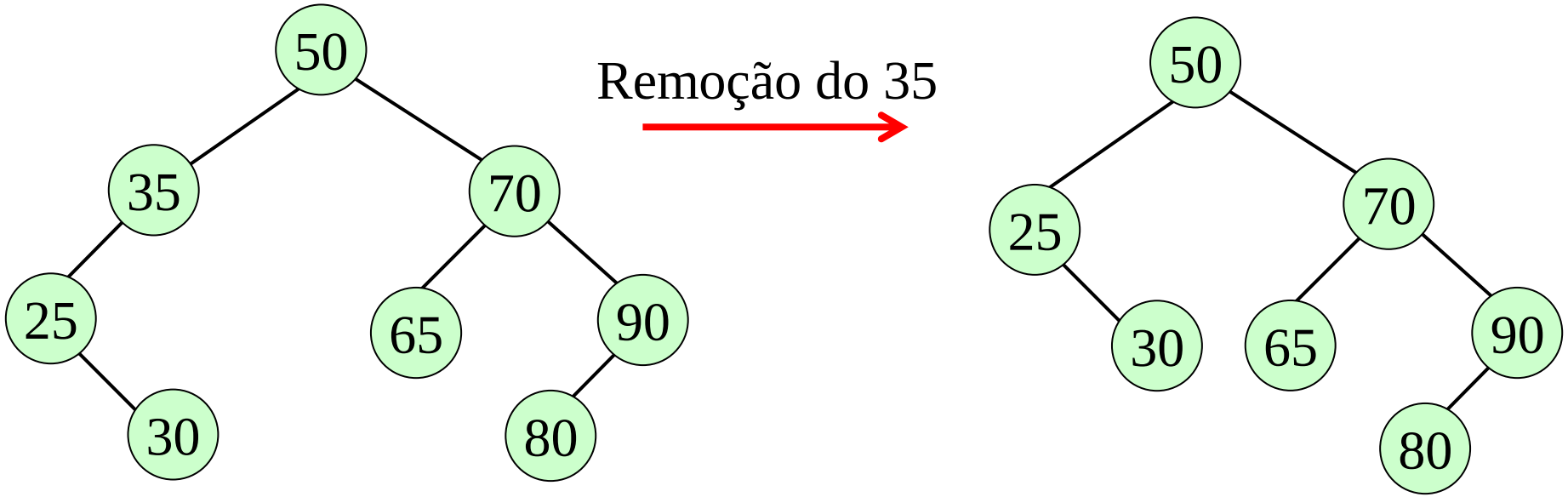
Caso 1. Deseja-se remover uma chave que está em uma folha.



Árvore Binária de Busca

Remoção

Caso 2. A chave removida não é uma folha, mas possui uma subárvore vazia.



Árvore Binária de Busca

Remoção

Caso 3. A chave removida não é uma folha e possui duas subárvores não vazias.

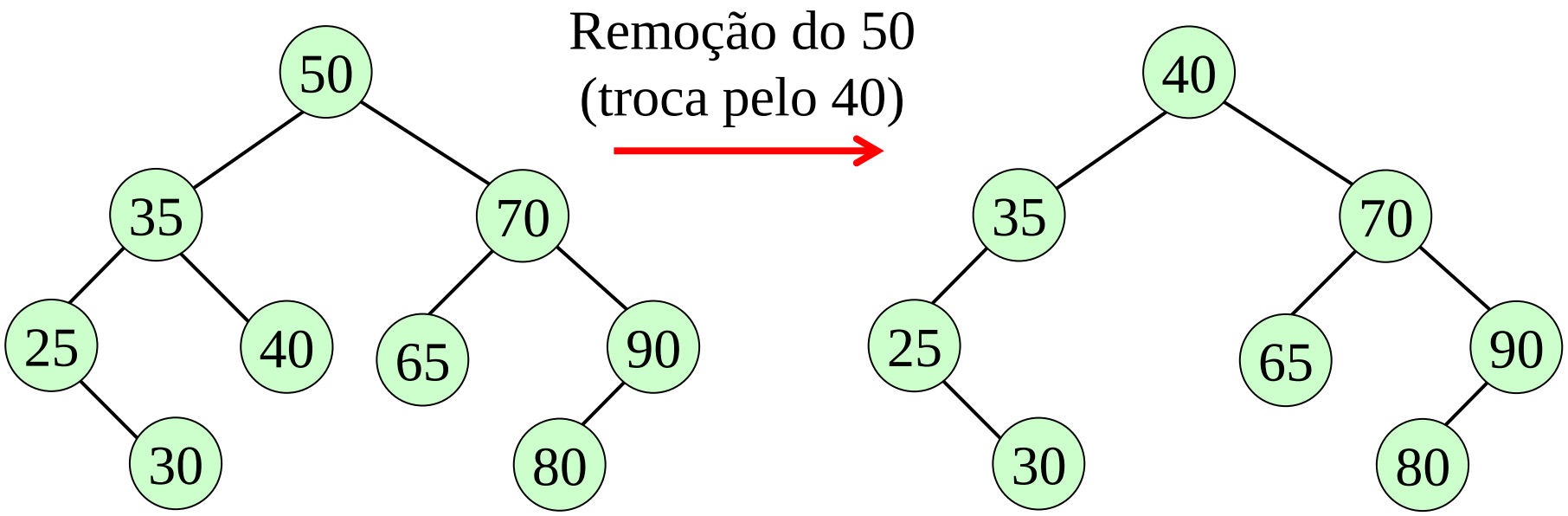
Neste caso, transformar em um dos casos anteriores.

Isto é feito substituindo o elemento a ser removido pelo maior elemento da subárvore esquerda ou o menor da subárvore direita.

Árvore Binária de Busca

Remoção

Caso 3. A chave removida não é uma folha e possui duas subárvores não vazias.



Árvore Binária de Busca

Remoção

Complexidade?

Árvore Binária de Busca

Remoção

Complexidade?

Dominada pela complexidade da busca

Exercício

Escrever o algoritmo de remoção em árvore binária de busca.



Árvore Binária de Busca

Comprimento de Caminho Interno

Para buscar uma chave, s_k, o algoritmo percorre um caminho da raiz até o nó s_k.

Árvore Binária de Busca

Comprimento de Caminho Interno

Para buscar uma chave, s_k, o algoritmo percorre um caminho da raiz até o nó s_k.

Dado um conjunto de n chaves $\{s_1,...,s_n\}$ devemos considerar quantas comparações no total o algoritmo efetuará para localizar todas as chaves.

Árvore Binária de Busca

Comprimento de Caminho Interno

Para buscar uma chave, s_k, o algoritmo percorre um caminho da raiz até o nó s_k.

Dado um conjunto de n chaves $\{s_1,...,s_n\}$ devemos considerar quantas comparações no total o algoritmo efetuará para localizar todas as chaves.

Para cada chave s_k o algoritmo fará nível(s_k) comparações.

Árvore Binária de Busca

Comprimento de Caminho Interno

Para buscar uma chave, s_k, o algoritmo percorre um caminho da raiz até o nó s_k.

Dado um conjunto de n chaves $\{s_1,...,s_n\}$ devemos considerar quantas comparações no total o algoritmo efetuará para localizar todas as chaves.

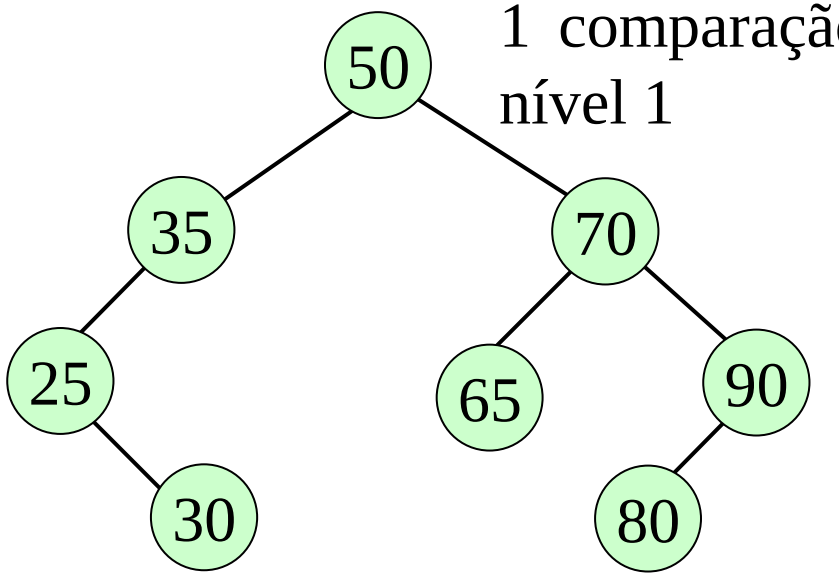
Para cada chave s_k o algoritmo fará nível(s_k) comparações.

O algoritmo fará $\sum_{k=1}^{n} nível(s_k)$ comparações.

Este valor é denominado *comprimento do caminho interno* de T, I(T).

Árvore Binária de Busca

Comprimento de Caminho Interno

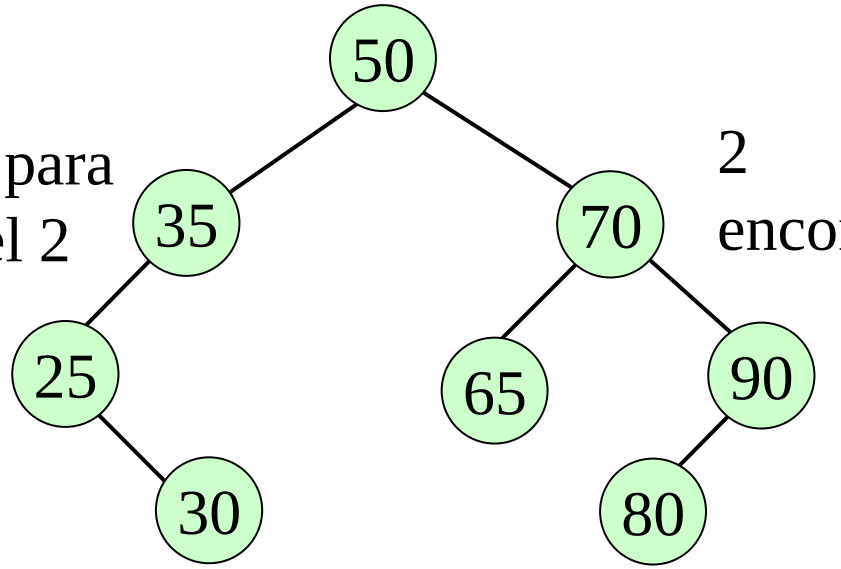


1 comparação para encontrar o 50 no nível 1

Árvore Binária de Busca

Comprimento de Caminho Interno

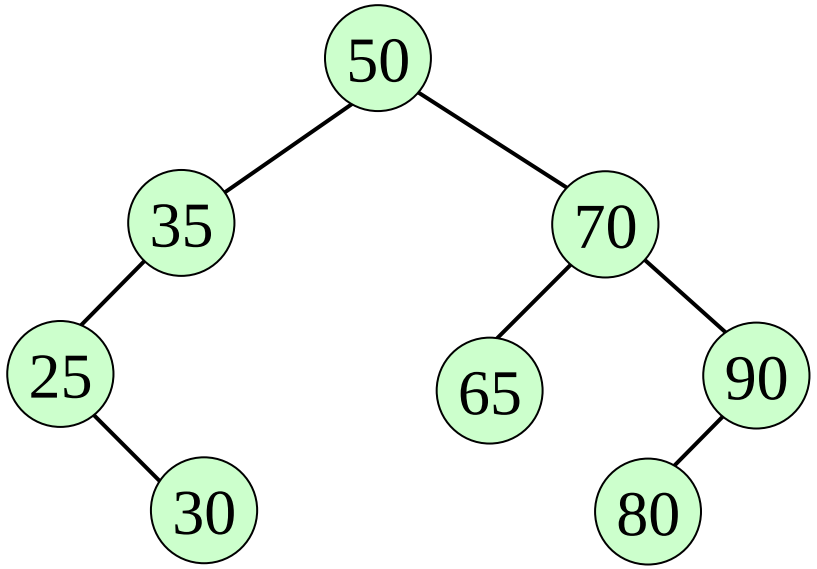
2 comparações para encontrar o 35 no nível 2



2 comparações para encontrar o 70 no nível 2

Árvore Binária de Busca

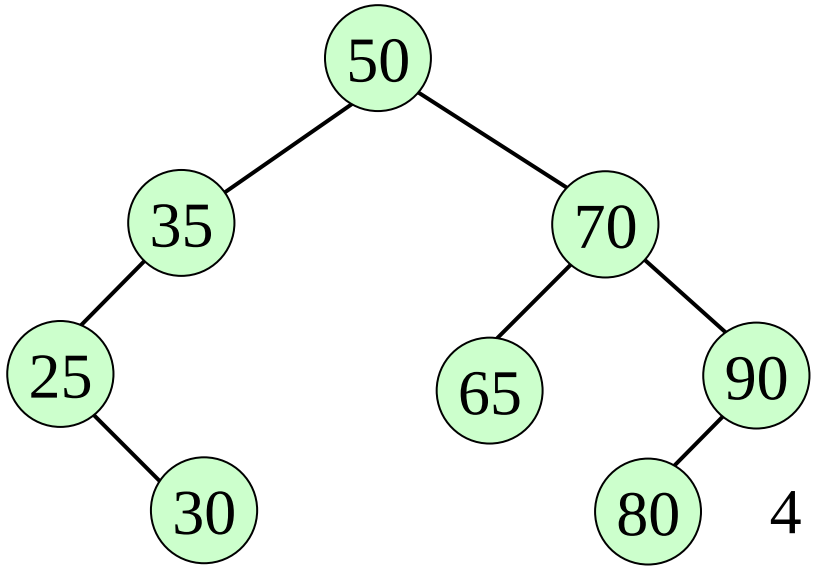
Comprimento de Caminho Interno



3 comparações para encontrar cada nó no nível 3: 25, 65 e 90

Árvore Binária de Busca

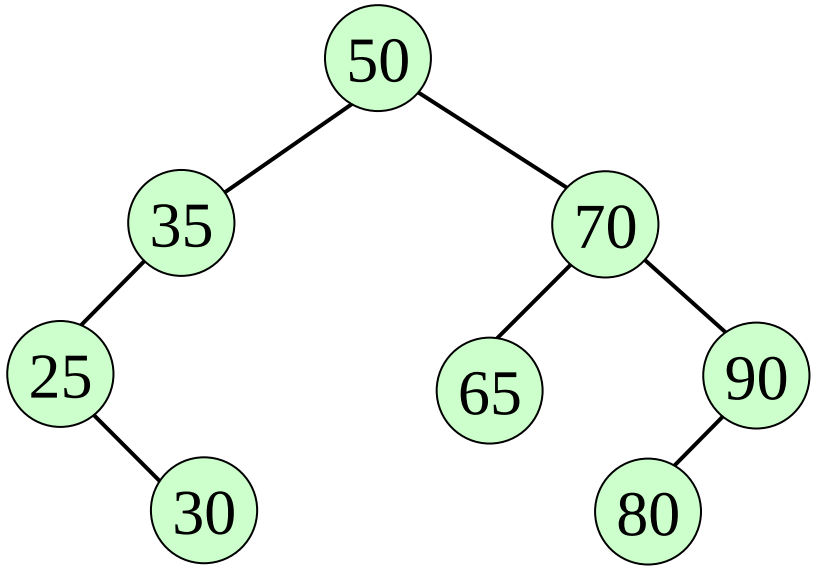
Comprimento de Caminho Interno



4 comparações para encontrar cada nó no nível 4: 30 e 80

Árvore Binária de Busca

Comprimento de Caminho Interno



$I(T) = 1(1) + 2(2) + 3(3) + 2(4) = 22$

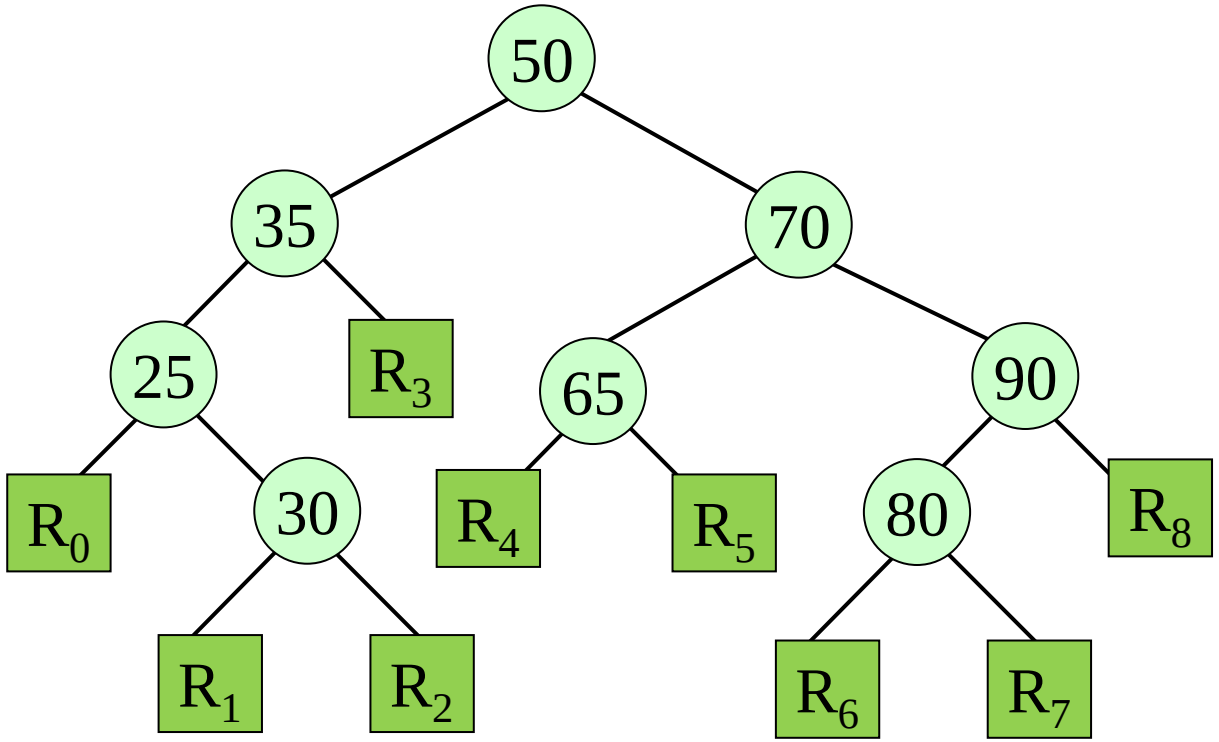
Número de comparações para localizar todas as chaves

Árvore Binária de Busca

Comprimento de Caminho Externo

Árvore Binária de Busca

Comprimento de Caminho Externo



Árvore com nós externos

Árvore Binária de Busca

Comprimento de Caminho Externo

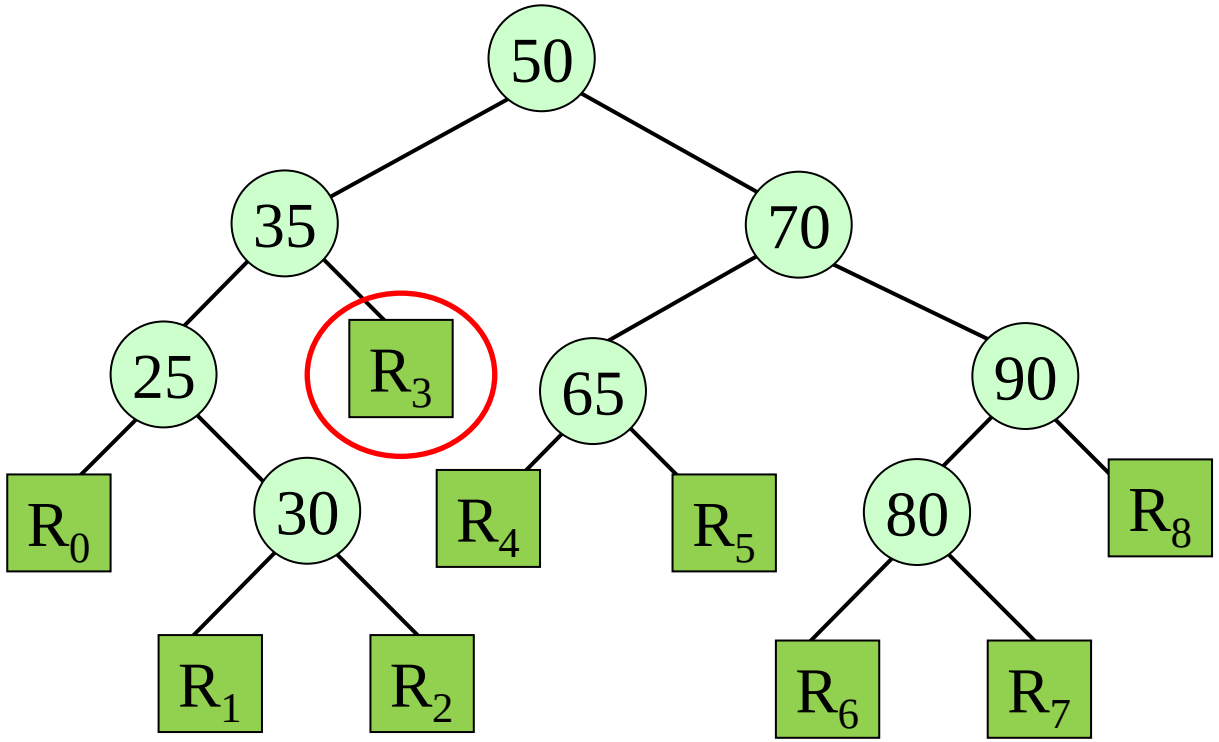
Quando se efetua uma busca sem sucesso de uma chave s_k o algoritmo percorre um caminho da raiz até um nó externo R_k e o número de comparações é nível(R_k) – 1.

Assim, analogamente ao comprimento de caminho externo define-se comprimento de caminho externo como

$$E(T) = \sum_{k=0}^{n} nível(R_k) - 1$$

Árvore Binária de Busca

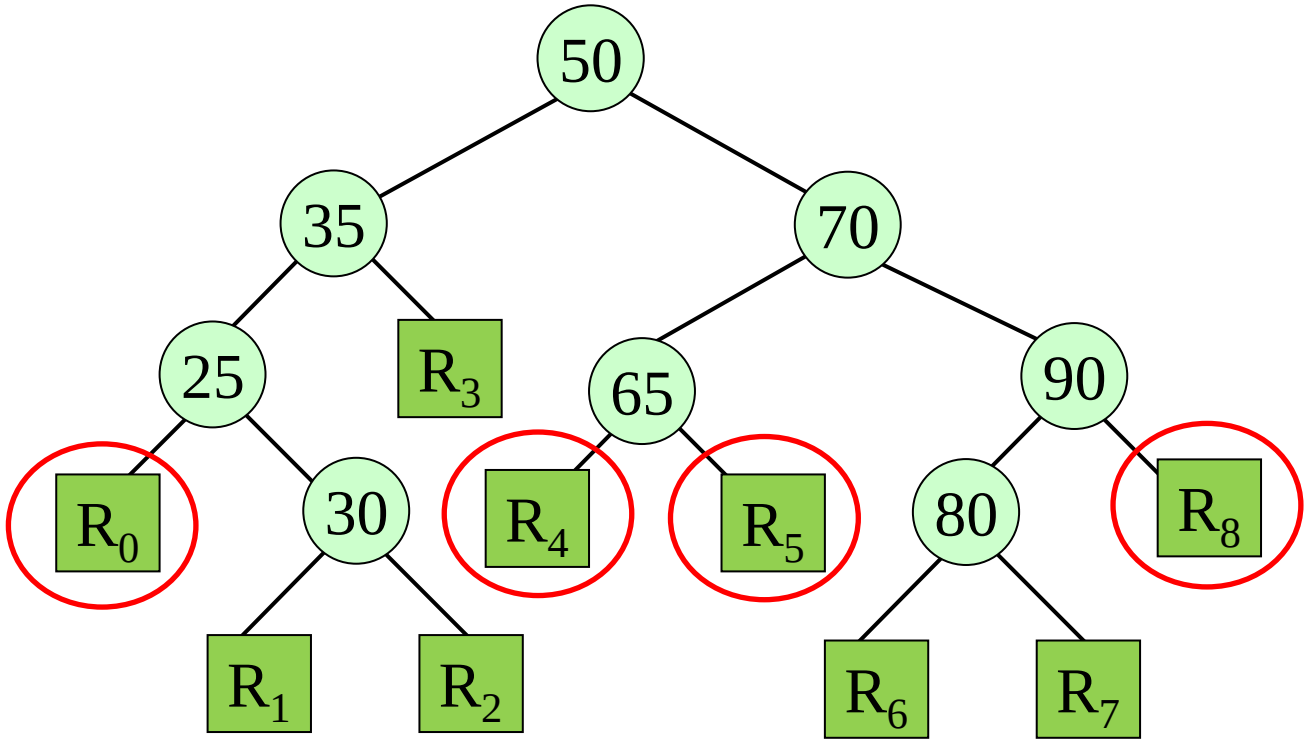
Comprimento de Caminho Externo



$$E(T) = 1(2) + 4(3) + 4(4) = 30$$

Árvore Binária de Busca

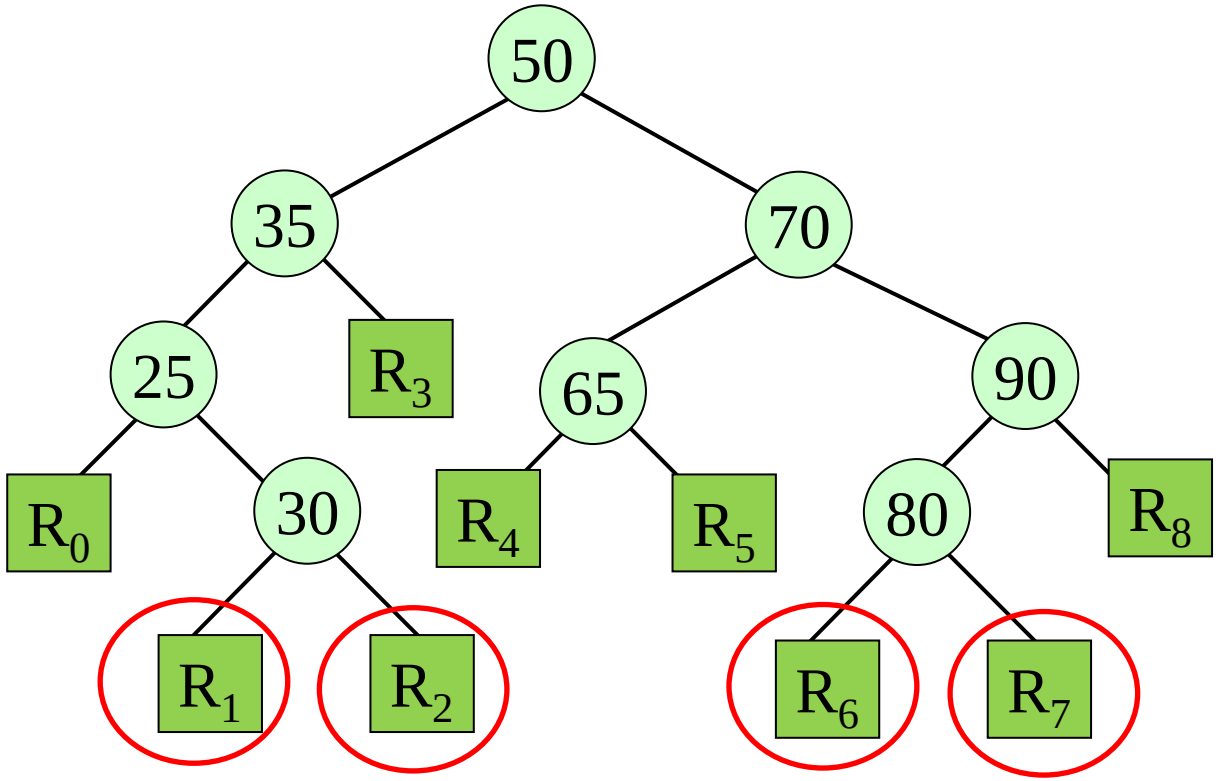
Comprimento de Caminho Externo



$$E(T) = 1(2) + 4(3) + 4(4) = 30$$

Árvore Binária de Busca

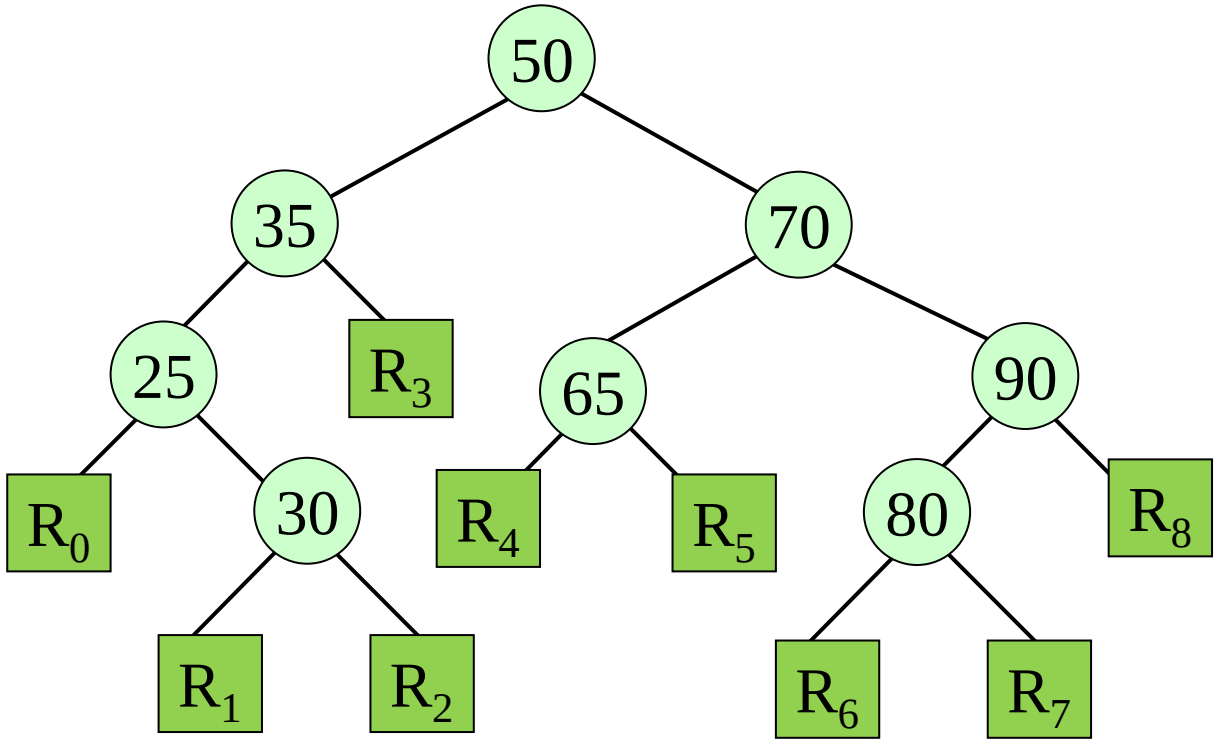
Comprimento de Caminho Externo



$$E(T) = 1(2) + 4(3) + 4(4) = 30$$

Árvore Binária de Busca

Comprimento de Caminho Externo



$$E(T) = 1(2) + 4(3) + 4(4) = 30$$

Exercício

Prove que

$$E(T) = I(T) + n$$



Árvore Binária de Busca

E(T) e I(T) servem para avaliar a “qualidade” da árvore binária de busca.

Os valores **I(T)/*n*** e **E(T)/(*n*+1)** representam o número médio de comparações que é necessário efetuar em operações de busca com e sem sucesso, respectivamente, quando todas as entradas são equiprováveis.

Exemplo: Na árvore dada como exemplo são necessárias 22/7 comparações para localizar uma chave e 30/8 comparações para concluir que uma chave não se encontra na árvore.

Naturalmente, deseja-se que I(*T*) e E(*T*) sejam mínimos.

Árvore Binária de Busca

E(T) e I(T) servem para avaliar a “qualidade” da árvore binária de busca.

Os valores **I(T)/*n*** e **E(T)/(*n*+1)** representam o número médio de comparações que é necessário efetuar em operações de busca com e sem sucesso, respectivamente, quando todas as entradas são equiprováveis.

Exemplo: Na árvore dada como exemplo são necessárias 22/7 comparações para localizar uma chave e 30/8 comparações para concluir que uma chave não se encontra na árvore.

Naturalmente, deseja-se que I(*T*) e E(*T*) sejam mínimos.

Árvore Binária de Busca

E(T) e I(T) servem para avaliar a “qualidade” da árvore binária de busca.

Os valores **I(T)/*n*** e **E(T)/(*n*+1)** representam o número médio de comparações que é necessário efetuar em operações de busca com e sem sucesso, respectivamente, quando todas as entradas são equiprováveis.

Exemplo: Na árvore dada como exemplo são necessárias 22/7 comparações para localizar uma chave e 30/8 comparações para concluir que uma chave não se encontra na árvore.

Naturalmente, deseja-se que I(*T*) e E(*T*) sejam mínimos.